

BRISKSNAPSHOT: A Live Demo of Join-Backed Semantic Windows for Streaming AI Pipelines

Anonymous Author(s)

Abstract

Fresh semantic state is becoming a first-class systems requirement for large-model applications. Operational copilots, streaming retrieval pipelines, and long-running agents all need intermediate vector state that can be updated continuously and consumed immediately by downstream reasoning steps. However, today this state is often reconstructed outside the runtime that observes the stream, making the resulting application behavior difficult to explain, reuse, and demonstrate. We present BRISKSNAPSHOT, a live demonstration of a unified streaming AI stack that keeps semantic-window state inside the runtime while exposing compact, application-readable contracts to downstream services. In the demo, BRISKSNAPSHOT replays a multi-source vulnerability-intelligence stream, shows how orchestration operators hand events to a persistent semantic-window runtime, and visualizes how maintained runtime state becomes vulnerability snapshots, routing decisions, and security escalation signals. A deterministic prepared run groups 24 events from five source classes into six semantic clusters and 18 snapshot insights, giving attendees a concrete way to inspect the relationship between runtime state, bounded summaries, and LLM-facing application behavior.

CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **Information systems** → **Stream management**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

stream processing, vector databases, AI middleware, multicore systems, demonstration

1 Introduction

Large-model systems are increasingly assembled as continuous dataflows instead of isolated request-response programs. New observations arrive between prompts, retrieved evidence changes while a task is in progress, and operational events must be clustered before an agent or copilot can react. In each case, the application needs fresh intermediate state rather than stale batch snapshots. Existing stream processors provide scalable execution [1], and vector indexes provide efficient nearest-neighbor search [2, 3], but an end-to-end AI stack still needs a practical way to maintain join-backed semantic state under sliding windows and continuous insertions.

This demo addresses that gap with BRISKSNAPSHOT, a unified software stack that combines application-level dataflow orchestration with a multicore vector-stream runtime. The specific problem is interface reliability: how can one system accept live vector events, maintain streaming semantic state under a bounded window, evaluate each new event against that state with a real window join, and return stable, application-readable summaries without forcing the upper pipeline to manage low-level runtime details?

Another difficulty comes from full-stack coordination. Runtime state ownership, Python bindings, operator orchestration, and application-visible output must all agree on what the live semantic window means and when its join results should be consumed. Without such an agreement, upper-layer operators either reconstruct state on their own or depend on brittle runtime-specific callbacks, which makes live execution harder to reason about and harder to demonstrate.

Contributions. We present BRISKSNAPSHOT, a concrete system that exposes live semantic state as a first-class join-backed window service for LLM inference pipelines. First, we integrate a runtime-owned window service into the same boundary used by the orchestration flow. Second, we derive bounded cluster, neighbor, and novelty summaries from those runtime matches so that higher-level operators consume a stable application contract rather than raw engine internals. Third, we package the prepared demo run as a reproducible artifact, including a generated result figure derived from the real JSON output of the live execution. The result is a compact but fully operational demonstration of how multicore streaming machinery can be surfaced as application-visible AI middleware.

The rest of the paper proceeds from problem setting to system design and then to the live demonstration path. We first introduce the system-level motivation behind join-backed semantic windows, then summarize the main architectural path used by BRISKSNAPSHOT, and finally present the concrete scenario and takeaways shown in the demonstration.

2 Background and Problem Overview

The core problem behind BRISKSNAPSHOT is not simply fast vector search. The harder systems problem is how to maintain fresh semantic state under continuous updates and expose that state to higher-level operators in a form that is stable enough for application logic. In a live AI pipeline, the state owner, the export interface, and the orchestration path all matter. If any of these are poorly chosen, the system becomes difficult to reason about, difficult to demonstrate, and difficult to reuse.

Our implementation makes this challenge visible because the operator layer, service layer, Python binding layer, and multicore runtime inside BRISKSNAPSHOT must all agree on what a “semantic window” means. This requirement leads to four practical design constraints.

First, the runtime must own the authoritative window state. If upper-layer operators reconstruct the window independently, they can drift from the true runtime view under future scheduling, reordering, or backpressure changes. Second, the service boundary must survive repeated use. Repeated execution requires reset behavior, cleanup behavior, and long-lived runtime ownership. Third, the exported state must have the right granularity. Exporting only final alerts hides too much of the semantic state, while exporting full internal structures leaks implementation details. Fourth, the

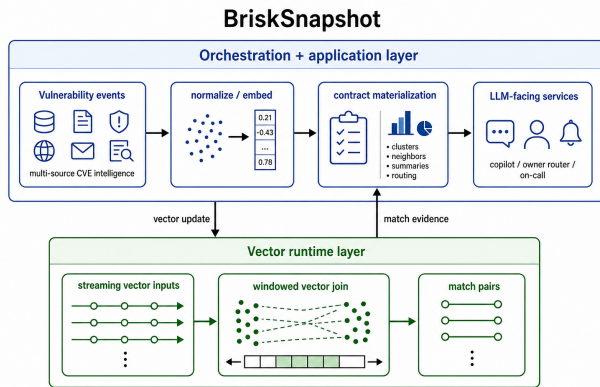


Figure 1: System overview of BRISKSNAPSHOT. The orchestration and application layer sends vector updates into a persistent runtime, receives match evidence back, and then materializes bounded contracts for LLM-facing services.

system must remain observable. Runtime behavior, exported state, and final application output should remain easy to relate within one execution.

These requirements shaped the current BRISKSNAPSHOT design. We therefore keep runtime ownership inside a persistent join runtime, use real window join to discover semantic matches, and expose bounded summaries rather than raw engine structures. This focus explains why the paper centers join-backed semantic windows rather than a broader collection of streaming operators.

3 System Overview

Figure 1 summarizes the architecture we implemented. The front half of BRISKSNAPSHOT is an application pipeline built from a batch source, a normalization step, a window-join step, an insight derivation step, and a sink. The critical operator is the window-join step: it invokes a runtime-local endpoint registered through the system’s service boundary. That service owns a long-lived semantic-window runtime, backed by streaming inputs and one continuously running execution graph. This control path lets BRISKSNAPSHOT expose live semantic state to higher-level operators while keeping runtime state ownership in one place.

3.1 Execution Path

The execution path is intentionally narrow. A prepared batch source replays the vulnerability-intelligence stream, the normalization step resolves each record into one uniform event type, and the window-join step forwards that event to the long-lived runtime through the local service registry. This design keeps orchestration logic simple: the upper pipeline never allocates short-lived runtime objects, and it never reconstructs window state outside the runtime. Instead, each event traverses one control path from source, to runtime update, to bounded summary, to insight derivation.

This path also matches what the audience sees in the live demo. The operator graph is small enough to explain quickly, but every stage performs real work. The source and map stages run in the

orchestration layer, the stateful update and window join happen inside the runtime layer, and the final sink emits the same JSON artifact that is later summarized in Figure 4. The chapter therefore describes one cohesive system path rather than a bridge between two independent products.

3.2 Join Runtime and State Materialization

Inside the join runtime, BRISKSNAPSHOT uses native streaming operators rather than a Python-side similarity scan. At service startup, the adapter creates one persistent runtime with dual streaming inputs, a windowed similarity join, and a sink that records emitted join pairs. For each arriving event, the orchestration layer appends a record to the runtime inputs and reads the newly emitted matches from that same running graph. Those matches return as vector evidence; the orchestration layer then folds them into bounded neighbor, cluster, and novelty summaries so that upper-layer operators receive a stable application contract while similarity discovery remains inside the streaming engine.

The design goal is to make the runtime the single owner of semantic matching. After each insertion, the adapter materializes the current active window, invokes the join operator against that bounded set, and derives a window summary containing cluster count, source breakdown, latest event identifier, nearest neighbors, and per-cluster statistics. In the current prototype, cluster construction follows connected components over the returned join matches, and the window summary is sorted by cluster size and average severity before it is exposed to the application. This gives the upper pipeline a compact state summary without exposing lower-level runtime structures.

The trade-off is deliberate. We do not export mutable internal indexes or rely on Python-level callbacks to represent authoritative similarity state. Instead, we keep match discovery inside the runtime and export only a bounded summary derived from the join results. This choice keeps the demo deterministic and debuggable while still leaving a real multicore runtime on the critical path.

This design matters because it removes the system’s correctness dependency on a Python-side reimplement of similarity discovery. The live path still uses real ingestion, a bounded active window, and deterministic summary generation, but the match computation itself remains inside the runtime. That trade-off keeps a real multicore runtime in the loop while preserving predictable and repeatable behavior.

3.3 Insight Materialization

The last part of the chapter is where BRISKSNAPSHOT turns runtime state into application-visible evidence. For every event, the adapter computes nearest neighbors from the join result, derives a novelty score from the top similarity, and identifies the event’s active cluster in the current summary. The downstream insight stage then applies explicit rules to this state: a cluster becomes a correlated incident when it contains at least three events, has average severity at least 0.70, and spans at least two sources; an event becomes an emerging pattern when its novelty score is at least 0.18 and its severity is at least 0.95. These rules are simple, but they are sufficient to show how join-backed semantic state can drive concrete next-step decisions.

Table 1: Responsibilities across the BRISKSNAPSHOT stack.

Layer	Responsibility
BRISKSNAPSHOT operators	Event normalization, service invocation, insight derivation, final sink output
Window-join access layer	Runtime ownership, reset and cleanup, stable application-facing summary access
Python binding	Record ingestion, join invocation, and result handoff into the application process
Join-backed window engine	Active-window materialization, bounded join execution, streaming state ownership

The architecture therefore illustrates a useful software boundary for AI infrastructure. Rather than forcing high-level operators to embed vector-state logic directly, BRISKSNAPSHOT keeps high-frequency state updates and window joins inside its runtime engine and exposes just enough summarized state to downstream operators through a stable system contract. The benefit is not only modularity. It also makes the effect of runtime parameters legible at the application layer: changing similarity threshold, active-window size, or event order does not just perturb an internal data structure, it changes which incidents are correlated, which anomalies remain novel, and what evidence the next inference stage receives.

Table 1 summarizes this division of responsibility. It makes clear that BRISKSNAPSHOT is one software stack with distinct internal roles rather than two separately described subsystems.

4 Demonstration Scenario

The demo section is written as a live walkthrough rather than as a conventional benchmark section. Recent systems demonstration papers typically use this part of the paper to make three things concrete: the scenario an attendee will see, the interface controls used during the session, and a small set of prepared results that can be reproduced without relying on live improvisation. We follow that structure for BRISKSNAPSHOT. The live path is intentionally compact, while the prepared evidence expands the same replay templates to larger workloads for scale and sensitivity discussion.

The live scenario replays 24 public security reports that resemble what an operations team might receive during a busy disclosure week. The reports come from five familiar categories: a national vulnerability catalog, vendor security advisories, public exploit warnings, emergency coordination alerts, and practitioner research notes. Instead of asking attendees to remember product acronyms, the demo groups the stream into six plain-language incident families: a firewall gateway flaw, a supply-chain backdoor, a remote-login service race, a web runtime bug, a VPN gateway flaw, and a build-server authentication bypass. For each arriving report, BRISKSNAPSHOT keeps a compact semantic representation inside the vector runtime, returns the matching evidence to the orchestration layer, and then hands a bounded contract to an LLM-facing response service. The

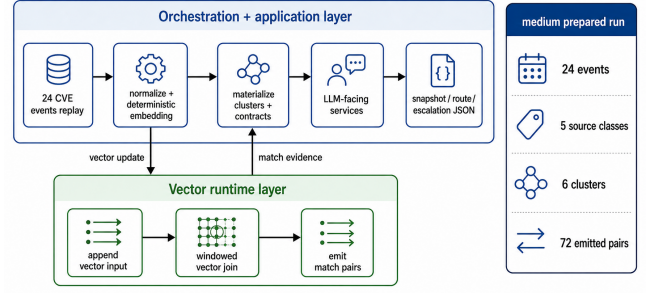


Figure 2: Live demonstration flow. The prepared replay sends vector updates into the runtime, receives match evidence back, and materializes snapshot, routing, and escalation outputs in the orchestration layer.

audience therefore sees both the runtime evidence and the final application effect: a grounded incident explanation, a routing decision, or an escalation note.

4.1 Live Walkthrough

The walkthrough follows one end-to-end path. First, the presenter starts the replay and selects the snapshot contract scenario. The interface highlights the current event, its source, and its deterministic embedding step. Second, the presenter follows the vector update into the runtime lane. This is the only layer that appends vector records and performs the windowed vector join. Third, the UI shows the evidence that returns upward: emitted match pairs, nearest neighbors, novelty, and the active cluster. Finally, the orchestration layer materializes a bounded contract consumed by application services. The same maintained state is then used to explain snapshot summaries, owner routing, and security escalation.

This order fixes two weaknesses in the earlier demo UI. The prior view displayed many cards, but the data movement was implicit; it was easy to miss that the runtime only handled vector state while application and LLM-facing services lived above it. The revised interface therefore promotes a dataflow trace above the detailed panels. It also folds repetitive output cards so that the paper screenshot shows representative contracts rather than a long log of near-identical insights.

4.2 Prepared Evidence and Data Scale

The 24-event walkthrough is deliberately small because it must be explainable event by event. It is not meant to be the only data shown in the paper. The artifact now includes deterministic expanded replay tiers created from the same public-event templates by repeating each event with a unique identifier, shifted timestamp, and replay metadata. This gives the demo a reproducible way to discuss scale without changing the conceptual pipeline shown in Figure 2.

The prepared variants remain the most useful experiments for a demo paper because they expose visible system behavior rather than isolated microbenchmarks. Raising the similarity threshold fragments the final state and suppresses correlated outputs. Reducing active-window size removes older vulnerability families from

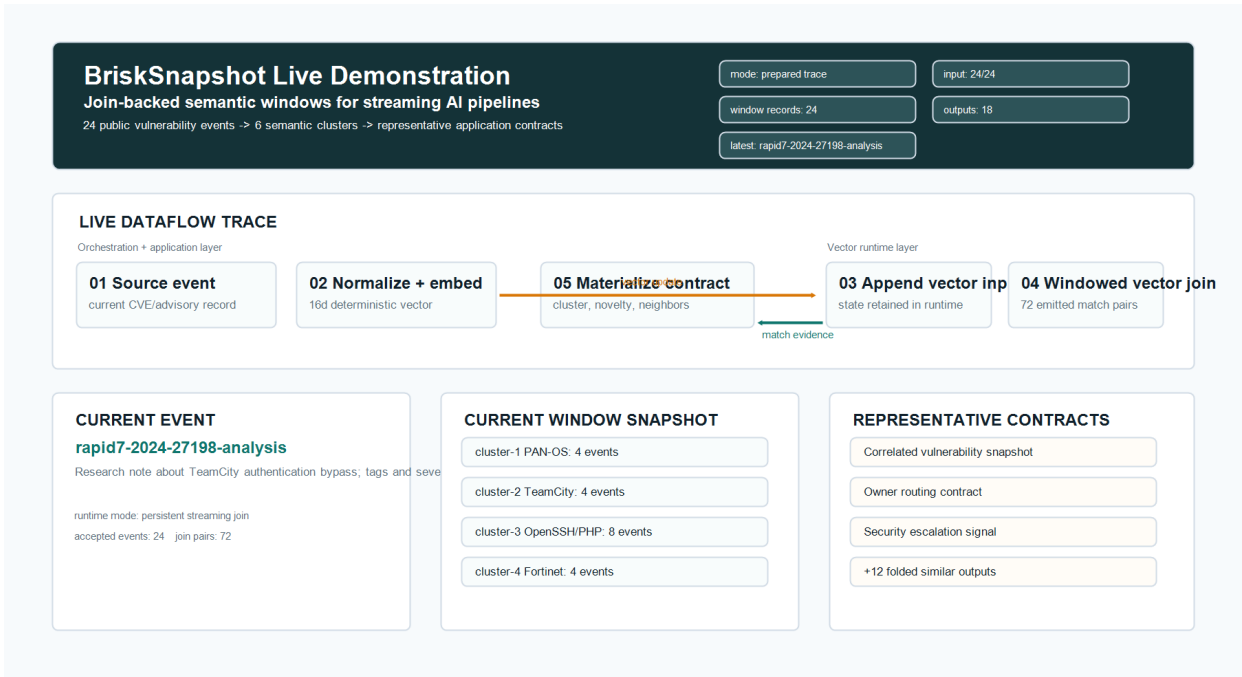


Figure 3: Demonstration interface view after the UI revision. The top trace separates the orchestration/application layer from the vector runtime layer so attendees can follow vector updates downward and match evidence upward before inspecting clusters and contracts.

Table 2: Attendee-facing panels in the live walkthrough.

Panel	What it demonstrates
Replay control	Deterministic start, reset, import, and live/backend connection state
Dataflow trace	Vector update enters the runtime; match evidence returns to orchestration
Current event	Source, tags, severity, and per-step latency for the active replay item
Runtime metrics	Retained records, emitted pairs, accepted events, and current runtime mode
Snapshot/contracts	Cluster membership and representative application-readable outputs

the exported state and shrinks active source coverage. Reversing the same stream preserves the topic families but changes the latest event and the order in which application-visible insights are emitted. Together, these runs expose semantic strictness, retention, and order sensitivity as concrete system-level trade-offs.

Figure 4 summarizes one deterministic prepared run. The revised figure emphasizes what the audience can understand from the run rather than only showing bar heights: the source categories, runtime evidence, six incident families, and the three application effects exposed to the LLM-facing service. The current implementation produces six clusters and 18 contracts from the 24-event

Table 3: Prepared replay tiers used by the demo and paper.

Tier	Events	Families	Role
Live walkthrough	24	6	Step-by-step UI narration
Extended replay	240	60	Parameter sensitivity and snapshot stability
Scale replay	2,400	600	Runtime handoff and retained-window stress
Long replay	24,000	6,000	Offline prepared evidence for the paper

walkthrough stream. Each cluster corresponds to a plain-language incident family, and the companion routing and escalation paths export contracts over the same maintained state.

This interaction model links parallel runtime structure to application-visible behavior without requiring a large deployment or a specialized dataset. The UI screenshot belongs in the paper because it shows the audience experience: controls, dataflow direction, runtime evidence, and final contracts are all visible at once. The scale table then prevents the paper from appearing to rely only on a toy data sample.

Table 4: Observed results of prepared variants executed on the live demo path.

Variant	Runtime setting	Observed result
Medium re-play	window=28, thresh-old=0.985	Final summary: 6 clusters of size 4; snapshot output: 18 correlated insights
Aggressive threshold	window=28, higher thresh-old	Final summary fragments into more smaller clusters; correlated snapshot output drops
Shorter active window	window=12, thresh-old=0.985	Older vulnerability families leave the exported state; active source coverage shrinks
Reordered stream	window=28, thresh-old=0.985, reversed order	Topic families remain visible, but latest event and insight emission order change

choosing a summary granularity, and providing an orchestration path that keeps the full stack understandable under live execution. The resulting system offers a compact but meaningful example of how parallel stream-processing mechanisms can be surfaced as practical AI middleware. An immediate next step is to expand the prepared scenario set and evaluate how active-window size, similarity thresholds, and arrival-order changes affect the quality and timeliness of exported evidence for downstream inference services.

References

- [1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, Sam Whittle, Robert Wilfong, and Lukasz Zajac. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.
- [2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [3] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

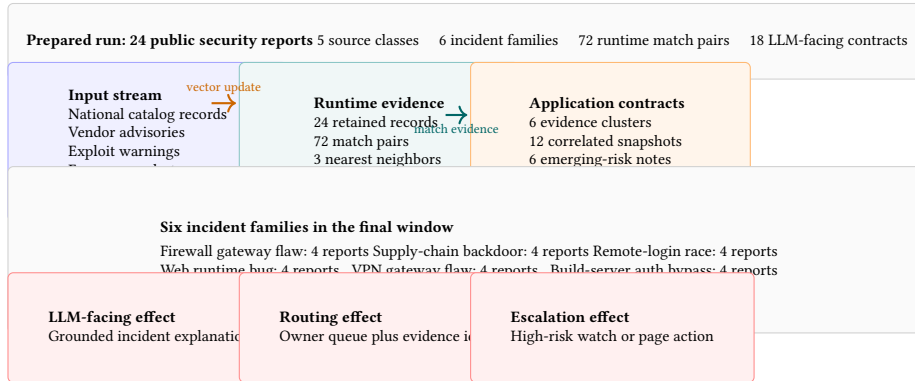


Figure 4: Result summary of one deterministic prepared BRISKSNAPSHOT run. The figure connects input reports, runtime evidence, incident families, and LLM-facing application effects.

5 Conclusion

This paper presents an ICPP demo built around BRISKSNAPSHOT, a concrete system that unifies streaming orchestration, vector-state maintenance, and join-backed semantic summaries. The central idea is to treat live vector-state management as a first-class system capability rather than as an isolated kernel or an external component. The current implementation already demonstrates this path on a persistent semantic-window runtime using a deterministic multi-source vulnerability-intelligence stream. In the prepared run used throughout the demo, BRISKSNAPSHOT turns 24 events from five source classes into six semantic clusters and 18 snapshot insights while keeping the summary path stable and application-readable.

More broadly, the demo matters because it turns interface design for parallel runtimes into an application-visible systems problem. BRISKSNAPSHOT shows that multicore stream processing is not only about accelerating kernels; it is also about choosing a state owner,